DSA Midterm Complexity Cheat Sheet

Three-page print reference | All four Lab 4 versions

Meaning: $T(n)$ measures how work grows; $S(n)$ measures memory. **Auxiliary space** excludes existing input storage; mention output and recursive call stack separately. Default table figures are worst-case unless marked best, average, or amortized. **Growing order:** $1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$.

1. Sorting

Sort	Best	Average	Worst	Aux.	Stable
Bubble (early exit)	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Yes
Bubble (no early exit)	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Yes
Insertion	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Yes
Selection, usual swap	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	No

Trace invariant: bubble fixes largest suffix; insertion grows sorted prefix; selection fixes smallest prefix. Array of n entries still occupies $\Theta(n)$ *stored* memory.

2. Searching

Search	Best	Worst	Extra memory / condition
Linear search	$\Theta(1)$	$O(n)$	$O(1)$; unsorted is fine
Binary, iterative	$\Theta(1)$	$O(\log n)$	$O(1)$; sorted indexed array
Binary, recursive	$\Theta(1)$	$O(\log n)$	$O(\log n)$ call stack
First/last occurrence binary	$O(\log n)$	$O(\log n)$	$O(1)$ iterative
Binary on answer	Varies	$O(C \log R)$	Monotone check costs C

Binary search trace (target 10): $[2, 4, 6, 8, 10, 12, 14]$; $(low, high, mid) = (0, 6, 3) \rightarrow (4, 6, 5) \rightarrow (4, 4, 4)$: found. Linked lists cannot access the midpoint in $O(1)$.

3. Arrays and sliding windows

Operation / pattern	Time	Space / condition
Array / `ArrayList` read or set index	$O(1)$	Input storage $O(n)$
Linear scan / count / print	$O(n)$	$O(1)$ auxiliary
`ArrayList.add` at end	Amort. $O(1)$	Resize once: $O(n)$ time
Insert/delete at front or middle	$O(n)$	Shifts following positions
Fixed-width window (running sum)	$O(n)$	$O(1)$ auxiliary
Naive sum/max of every k-window	$O((n-k+1)k)$	$O(1)$ auxiliary
Window maximum (monotonic deque)	$O(n)$	$O(k)$ deque
Two-forward-pointer window	$O(n)$	Condition must be valid
Build prefix sums / query afterward	$O(n)$ / $O(1)$	$O(n)$ precomputed array

4. Count loops correctly

Pattern	Time	Explanation
Two consecutive loops, lengths n, m	$O(n+m)$	Add independent costs
Nested loops, dimensions n, m	$O(nm)$	Multiply repeated work
Triangular loop $j < i$	$O(n^2)$	$0 + 1 + \cdots + (n-1)$
Doubling/halving counter	$O(\log n)$	Number of size reductions
Monotonic indices, each pushed/popped once	$O(n)$	Amortized total, despite `while`

Big O: upper bound; Θ: tight bound; Ω: lower bound. **Best/average/worst** is a separate classification. **Amortized** means average per operation over a sequence, not a probabilistic guarantee.

Stacks, Queues, Deques, and Heaps

5. Stack family (LIFO)

Variant	Push / pop / peek	Storage
Fixed array stack	$O(1)$ / $O(1)$ / $O(1)$	$O(K)$ capacity
Growable array stack	Push amort. $O(1)$ (resize $O(n)$); pop/peek $O(1)$	$O(n)$
Linked stack (head node)	$O(1)$ / $O(1)$ / $O(1)$	$O(n)$ nodes
Java `ArrayDeque` stack	Amort. $O(1)$ / amort. $O(1)$ / $O(1)$	$O(n)$
Min stack	Push/pop/`getMin` $O(1)$	$O(n)$ min history
Undo/redo using two stacks	Edit/undo/redo $O(1)$	$O(n)$ history

Stack-based algorithms: bracket matching $O(n)$ time/$O(n)$ extra; reversing with extra stack $O(n)/O(n)$; next greater element with monotonic stack $O(n)/O(n)$. Searching a stack by value costs $O(n)$.

6. Queue family (FIFO unless priority)

Variant	Enqueue / dequeue / peek	Storage
Shift-on-dequeue array queue	$O(1)$ / $O(n)$ / $O(1)$	$O(K)$ fixed
Circular-array queue	$O(1)$ / $O(1)$ / $O(1)$	$O(K)$ fixed
Head+tail linked queue	$O(1)$ / $O(1)$ / $O(1)$	$O(n)$ nodes
Java `ArrayDeque` queue	Amort. $O(1)$ / amort. $O(1)$ / $O(1)$	$O(n)$
Double-ended queue (deque)	Add/remove/peek at either end $O(1)$ for fixed ring or linked; amortized for growable	$O(K)$ or $O(n)$
Binary-heap priority queue	$O(\log n)$ / $O(\log n)$ / $O(1)$	$O(n)$ heap

7. Heap and monotonic-structure details

Operation / algorithm	Time	Extra storage
Binary heap peek, size, empty	$O(1)$	$O(1)$
Binary heap insert, extract	$O(\log n)$	$O(1)$ for array heap internal step
Heap arbitrary contains/remove by value	$O(n)$ worst	$O(1)$ extra
Bottom-up heapify from n items	$O(n)$	$O(1)$ extra possible
Monotonic stack: process n values	$O(n)$ total	$O(n)$ indices
Monotonic deque: max of k-windows	$O(n)$ total	$O(k)$ candidate indices
Print all items from a container	$O(n)$	$O(1)$ iterator + output

Different names: circular array queue $\neq$ circular linked list. Deque $\neq$ priority queue. A heap does *not* guarantee FIFO for equal priorities unless you store arrival order as an additional comparison key.

8. Essential complexity conditions

Claim	Needed assumption
Append to linked list: $O(1)$	Tail reference maintained
Remove known linked node: $O(1)$	Doubly linked node known, or singly predecessor known
Queue enqueue/dequeue: $O(1)$	Circular-index array or correct head/tail links
Stack push: amortized $O(1)$	Geometrically growing array
Priority-queue insertion: $O(\log n)$	Binary heap representation
Sliding maximum: $O(n)$	Monotonic deque, not repeated k-scans
Binary search: $O(\log n)$	*Sorted*, random-access data

Read time and space separately: Heap insertion runs in $O(\log n)$ time while the heap stores $O(n)$ items. Linked enqueue runs in $O(1)$ while the linked queue stores $O(n)$ nodes.

Linked Lists, Recursion, and Exam Comparisons

9. Linked list operations (H+T = head and tail retained)

Operation	Singly H+T	Doubly H+T	Circular singly + tail
Access index / find by value	$O(n)$	$O(n)$	$O(n)$
Prepend / append	$O(1)$ / $O(1)$	$O(1)$ / $O(1)$	$O(1)$ / $O(1)$
Delete first / last	$O(1)$ / $O(n)$	$O(1)$ / $O(1)$	$O(1)$ / $O(n)$
Delete known interior	$O(1)$ with previous node	$O(1)$ with node	$O(1)$ with previous node
Delete by value	$O(n)$	$O(n)$	$O(n)$
Print/traverse / reverse iteratively	$O(n)$ / $O(n)$	$O(n)$ / $O(n)$	$O(n)$ / $O(n)$
Stored data space	$O(n)$	$O(n)$	$O(n)$

Circular doubly: add/remove either end or delete a known node $O(1)$; search $O(n)$; storage $O(n)$. **Singly head-only:** tail append $O(n)$. **Floyd cycle detection:** time $O(n)$; extra $O(1)$. **Recursive reversal:** time $O(n)$; call-stack $O(n)$.

10. Quick recurrences

Recurrence	Time	Typical problem
$T(n) = T(n-1) + O(1)$	$O(n)$	Factorial, recursive sum
$T(n) = T(n/2) + O(1)$	$O(\log n)$	Recursive binary search
$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$	Divide and merge
$T(n) = T(n-1) + T(n-2) + O(1)$	$O(2^n)$ upper	Naive Fibonacci

11. Recursion memory, not just time

Algorithm	Time	Call-stack extra space
Factorial / recursive array sum	$O(n)$	$O(n)$
Recursive binary search	$O(\log n)$	$O(\log n)$
Naive two-branch Fibonacci	$O(2^n)$	$O(n)$
Memoized Fibonacci	$O(n)$	$O(n)$ cache + frames
Iterative array sum	$O(n)$	$O(1)$

Stack-depth principle: Count the maximum number of active recursive calls, *not* the total number of calls. Java does not guarantee tail-call elimination.

12. How to compare two algorithms

1. Name input sizes n, m, k and whether values are sorted.
2. Count work in loops and recursive branches.
3. Simplify dominant terms: $3n^2 + 5n + 2 = \Theta(n^2)$.
4. Report the same case: best, average, or worst.
5. Compare *both* time and auxiliary space.
6. Check preconditions, amortized/expected labels and actual input bounds.

13. Composed tasks

Task	Overall time
Insertion sort then one binary search	$O(n^2 + \log n) = O(n^2)$
Efficient $O(n \log n)$ sort + q binary searches	$O(n \log n + q \log n)$
Outer n loop, inner m loop	$O(nm)$
Two independent scans of sizes n, m	$O(n + m)$
n monotonic deque insertions/removals	$O(n)$ total (amortized)

Large-input intuition: For $n = 10^4$, $n^2 = 10^8$ versus $n \log_2 n \approx 1.33 \cdot 10^5$. These are operation-growth comparisons, not measured seconds.